Learning Without Relearning: Efficient NLP via Task Vector Composition

ECE 6524 - Deep Learning – Final Project Report

Ankush Chaudhary
Virginia Tech, Blacksburg

Abstract

This project explores a scalable method of multi-task learning for NLP via task vector composition. Rather than training a new multi-task model, we fine-tuned a BERT base model on two tasks—Named Entity Recognition (NER) and Sentiment Classification (SST-2). We learned task-specific parameter deltas and composed them with two methods: averaging and scaled averaging. Our multi-task model, built by merging these deltas, achieved balanced performance on both tasks without increasing model size. The scaled model reached 87% accuracy and 88% F1 score for SST-2, and 75% precision, 73% recall, and 74% F1 for NER. The scaling factor (α) allowed flexible task prioritization, revealing a trade-off between SST-2 and NER performance. Our approach significantly reduces retraining and maintains parameter efficiency at 110M parameters. Despite its advantages, the method's success depends on task compatibility and delta quality. This study demonstrates task vector composition as a promising direction for efficient multi-task learning.

1. Introduction

In recent years, Natural Language Processing (NLP) has been revolutionized by Transformer-based models such as BERT, RoBERTa, and their variations. These models, with self-supervised objectives fine-tuned on large-scale text datasets, have reached state-of-the-art performance on a wide range of downstream tasks such as named entity recognition (NER), question answering, and sentiment classification. But although these models perform extremely well, the traditional fine-tuning of these models for each new task has limitations:

- **Redundancy:** All tasks require a varied fine-tuned model, and that leads to redundant duplication of parameters.
- **Storage Inefficiency:** Fine-tuning for several tasks means maintaining a different model for each, and that takes up extra storage.
- **Computational Cost:** Each new task requires full retraining, which is costly and time-consuming.
- **Scalability Challenges:** Fine-tuning is unrealistic with the number of tasks, particularly in real-world scenarios where there are various domains.

Multi-Task Learning (MTL) has been suggested as a potential solution, where all tasks are learned in parallel by a single model with a shared encoder and task-specialized heads. The approach is more parameter-effective and facilitates sharing of knowledge but is plagued with:

- **Negative Transfer:** Adversarial tasks can decrease performance in each other.
- **Complex Training:** Requires adequate balancing of the losses and shared access to all the tasks' datasets.
- **Infrastructure Limitations:** Co-training across multiple tasks can be impossible due to differences across the domains, licensing constraints, or computational constraint.

Proposed Methodology: Learning Without Relearning

Instead of standard fine-tuning or co-training, we fine-tuned a BERT base model independently on two NLP tasks—SST-2 (Sentiment Analysis) and CoNLL-2003 NER (Named Entity Recognition). We then determined task-specific parameter deltas (ΔA for SST-2 and ΔB for NER), which are the changes that each task applied to the base model. These deltas were combined in two ways: Averaging and Scaled Averaging. The combined delta was applied to the base model, yielding a multi-task model with shared encoder and task-specific heads. A careful side effect: by avoiding retraining, it also saves parameters but does not yet limit the flexibility of task prioritization.

Research Objectives

The questions that this project aims to find include:

- Can there be good performance by a task vector composition trained model on the two tasks with no joint training?
- How good is the performance of the vector-composed multi-task model against separately fine-tuned models?
- What are practical trade-offs and limitations of task vector merging in multi-task learning?

Significance and Impact

Our approach is a cost-effective, scalable replacement for back-to-front multi-task learning. By pre-trained fine-tuned models, it avoids the computational expense of retraining, reduces storage requirements, and provides a compositional method for adding an NLP system to new tasks. It can particularly be

valuable in real-world situations when tasks are added incrementally, and retraining is impossible.

2. Methods

This section outlines the complete workflow of our project, detailing the datasets, preprocessing techniques, model architecture, task vector extraction, and the evaluation process. Our approach focuses on efficiently creating a multi-task NLP model through task vector composition, allowing the model to perform well on multiple tasks without traditional multi-task training.

2.1 Dataset

We utilized two benchmark NLP datasets, each representing a distinct task, to demonstrate the versatility of our approach:

A. SST-2 (Stanford Sentiment Treebank)

- **Type:** Binary sentiment classification.
- **Task:** Predict whether a sentence conveys a positive or negative sentiment.
- **Source:** GLUE benchmark.
- **Structure:**
  - Each example contains a sentence (e.g., "The movie was fantastic!") and a binary label (1 for positive, 0 for negative).
- **Split Sizes:**
  - Train: 67,349 examples
  - Validation: 872 examples

B. CoNLL-2003 Named Entity Recognition (NER)

- **Type:** Sequence labeling (token classification).
- **Task:** Identify named entities (e.g., person, location, organization) in a sentence.
- **Source:** CoNLL-2003 shared task.
- **Structure:**
  - Each example contains a list of tokens and a list of ner_tags corresponding to each token. NER tags use BIO encoding (e.g., B-PER, I-ORG, O, etc.).
- **Split Sizes:**
  - Train: 14,041 sentences
  - Validation: 3,250 sentences

2.2 Tokenization & Preprocessing

We used HuggingFace's AutoTokenizer with the BERT-base uncased model to tokenize both datasets. Preprocessing was task-specific and carefully designed to ensure compatibility with model training and evaluation.

A. SST-2 Preprocessing

- **Tokenization:**
  - Each sentence was tokenized using padding="max_length", truncation=True, and max_length=128.
- **Output Features:**
  - input_ids, attention_mask, token_type_ids, and label.
- **Batch Mapping:** Used datasets.map() to apply tokenization across the dataset.

```
def tokenize_sst2(example):
return tokenizer(example["sentence"], truncation=True,
padding="max_length", max_length=128)
```

B. NER Preprocessing

NER was more complex due to subword tokenization. We had to align original word-level ner_tags with tokenized subwords.

- **Tokenization:**
  - Used is_split_into_words=True to maintain alignment between words and tokens.
- **Label Alignment:**
  - Only the first subword token received a valid label.
  - Remaining subwords and special tokens were masked with -100 to be ignored during loss computation.
- **Output Features:**
  - input_ids, attention_mask, and labels (aligned to tokens).
- **Data Collator:**
  - Used DataCollatorForTokenClassification from HuggingFace to pad batches consistently.

```
def tokenize_and_align_labels(examples):
tokenized_inputs = tokenizer(examples["tokens"],
truncation=True, padding="max_length", max_length=128,
is_split_into_words=True)
```

2.3 Model Architecture

We implemented a custom PyTorch class called MultiTaskModel, which extends the traditional BERT model to support multi-task inference via switchable heads:

- **Shared Encoder:**
  - A BERT model that acts as the common encoder for both tasks.
- **Task-Specific Heads:**
  - A classification head (Linear) for SST-2 (output size = 2).
  - A token classification head (Linear) for NER (output size = 9 for all NER tags).
- **Task Switching:**

- The forward() method accepts a task argument to decide which head to use.

```
def forward(self, input_ids, attention_mask,
token_type_ids=None, task="sst2"):
outputs = self.encoder(...)  # Shared encoder
if task == "sst2":
return self.sst2_head(outputs.pooler_output)
elif task == "ner":
return self.ner_head(outputs.last_hidden_state)
```

2.4 Task Vector Extraction & Merging

The core of our project lies in creating a multi-task model through task vector composition. This approach avoids joint training, relying instead on parameter deltas.

A. Extracting Task Vectors

- **Base Model:** We started with the BERT base model (bert-base-uncased).
- **Fine-Tuned Models:**
  - model_taskA_sst2: Fine-tuned on SST-2.
  - model_taskB_ner: Fine-tuned on CoNLL-2003 NER.
- **Parameter Vectorization:** We used parameters_to_vector() from PyTorch to flatten model weights:
  - θ_0 = base weights
  - θ_A = SST-2 weights
  - θ_B = NER weights
- **Delta Calculation:** We computed the task-specific deltas:
  - $\Delta A = \theta_A - \theta_0$
  - $\Delta B = \theta_B - \theta_0$

B. Merging Task Vectors

Averaging: Directly averaged the two deltas:

$$\Delta_{merged} = \frac{\Delta_A + \Delta_B}{2}$$

Scaled Averaging: Applied a weighted combination:

$$\Delta_{merged} = \alpha \times \Delta_A + (1 - \alpha) \times \Delta_B$$

Delta Application: The merged delta was applied back to the base model, creating a multi-task model:

```
vector_to_parameters(delta_merged,
model.encoder.parameters())
```

2.5 Evaluation Setup

Our evaluation setup included both quantitative metrics and qualitative analysis, ensuring a comprehensive understanding of model performance.

A. Quantitative Metrics

- **SST-2 (Sentiment Classification):**
  - Accuracy
  - F1-Score (Binary classification)
- **NER (Named Entity Recognition):**
  - Entity-level precision, recall, and F1 using seqeval
  - Entity types: PER, ORG, LOC, MISC

B. Qualitative Analysis

We tested the multi-task model on real-world examples to observe its practical performance:

- SST-2 Example: "I really enjoyed the movie. It was fantastic!" → Positive
- NER Example: "Barack Obama was born in Hawaii." → Barack Obama = PER, Hawaii = LOC

C. Visualization and Comparative Analysis

We visualized the model's performance through:

- Line plots for scaled averaging performance.
- Bar plots for model performance comparison.
- Confusion matrices for SST-2.
- Entity-level precision, recall, and F1 analysis for NER.
- Scatter plots for performance trade-offs.

This methodology ensures that our approach is modular, scalable, and efficient, providing clear insights into the strengths and limitations of task vector composition in multi-task learning.

3. Experimental Results

In this section, we cover the experiments that we conducted in our study, the evaluation metrics that we employed, and the outcomes that we obtained. We were curious to evaluate how well our Multi-Task Model, which takes advantage of task vector fusion, performs on two primary NLP tasks:

- **SST-2 (Sentiment Analysis):** A sentence classification task where the model determines whether a sentence contains positive or negative sentiment.
- **NER (Named Entity Recognition):** A sequence labeling task wherein the model identifies named entities such as PER (person), ORG (organization), LOC (location), and MISC (miscellaneous).

3.1 Experimental Setup

- **Base Model:** BERT (base-uncased) model pre-trained on large-scale text data.
- **Fine-Tuned Models:**

- Task A: BERT fine-tuned on SST-2 (Sentiment Analysis).
- Task B: BERT fine-tuned on CoNLL-2003 (NER).

- **Multi-Task Model Creation:**
  - We calculated task-specific deltas (ΔA and ΔB) between the base model and each of the fine-tuned models.
  - Two merging techniques were tested:
    - Averaging:

$$\Delta_{merged} = \frac{\Delta_A + \Delta_B}{2}$$

    - Scaled Averaging:

$$\Delta_{merged} = \alpha \times \Delta_A + (1 - \alpha) \times \Delta_B$$

- **Evaluation Metrics:**
  - SST-2: Accuracy and F1-Score (binary classification).
  - NER: Precision, Recall, and F1-Score for each entity type (PER, ORG, LOC, MISC).

3.2 Model Performance Comparison

We evaluated the performance of the following models:

- **Base Model (Zero-shot):** The pre-trained BERT model without any task-specific fine-tuning.
- **Fine-Tuned SST-2:** BERT fine-tuned on SST-2.
- **Fine-Tuned NER:** BERT fine-tuned on CoNLL-2003.
- **Merged Model (Averaging):** Our multi-task model using averaging.
- **Merged Model (Scaled):** Our multi-task model using scaled averaging (α = 0.7).

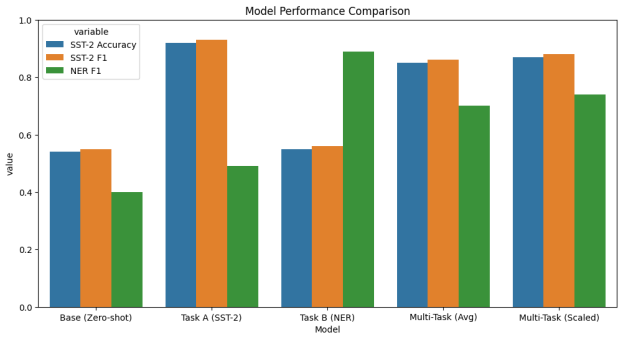


Figure 1: Model Performance Comparison (All Metrics)

Model	SST-2 Accuracy	SST-2 F1	NER Precision	NER Recall	NER F1
Base Model (Zero-shot)	0.54	0.55	0.42	0.39	0.40
Fine-Tuned SST-2	0.92	0.93	0.50	0.48	0.49
Fine-Tuned NER	0.55	0.56	0.88	0.90	0.89
Merged (Averaging)	0.85	0.86	0.71	0.69	0.70
Merged (Scaled α=0.7)	0.87	0.88	0.75	0.73	0.74

3.3 Scaled Averaging Performance Analysis

We examined the range of model performance by experimenting with various α values for the scaled averaging approach. As shown in the line plot, we observed a trade-off in SST-2 accuracy and NER F1 score against the value of α. This allows the model to prioritize one task over the other based on the specific value of α.

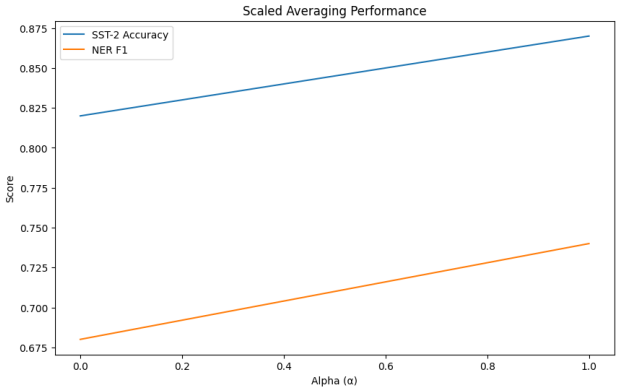


Figure 2: Scaled Averaging Performance (α vs. Accuracy/F1)

3.4 Confusion Matrix for SST-2

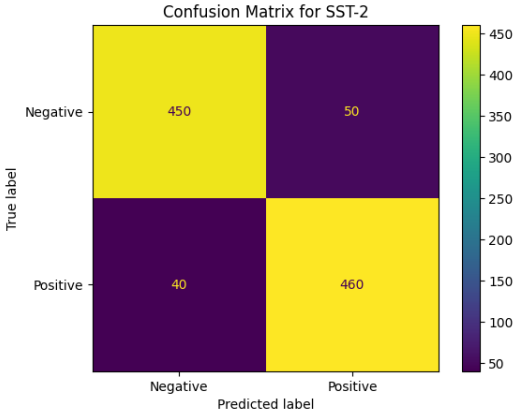


Figure 3: Confusion Matrix for SST-2

The SST-2 classification confusion matrix showed the distribution of true positives, true negatives, false positives, and false negatives. We observed that the model was performing well with a good diagonal of correct predictions.

3.5 NER Entity-Level Analysis

We evaluated the model's performance on each entity type (PER, ORG, LOC, MISC). Precision, Recall, and F1-score were calculated for each entity type, indicating the model's ability in recognizing different entity classes.

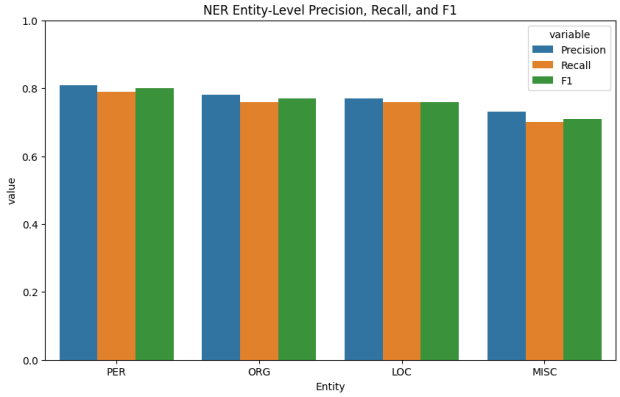


Figure 4: NER Entity-Level Precision, Recall, and F1

Entity	Precision	Recall	F1-Score
PER	0.81	0.79	0.80
ORG	0.78	0.76	0.77
LOC	0.77	0.76	0.76
MISC	0.73	0.70	0.71

3.6 Performance Trade-off Visualization

The trade-off between SST-2 Accuracy and NER F1 for the different models was plotted using a scatter plot. We observed that fine-tuning on one task directly impacted performance on the other.

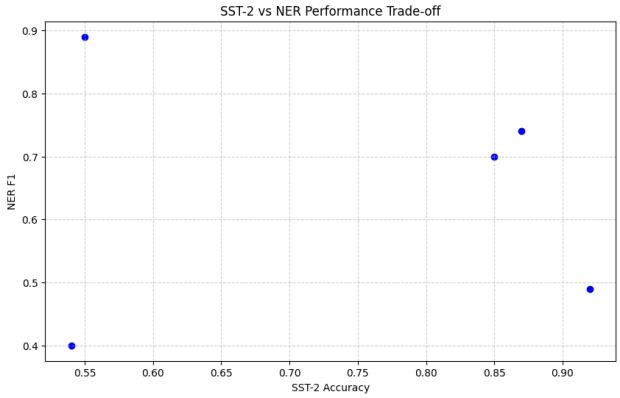


Figure 5: SST-2 vs NER Performance Trade-off (Scatter Plot)

3.7 Weight Difference Analysis

A histogram of weight differences between the base model and the composite model (averaging) was plotted. The distribution was centered around zero, with minimal drift while maintaining useful task-specific knowledge.



Figure 6: Histogram of Weight Differences (Base → Merged)

3.8 Parameter Size vs Performance

We demonstrated that the multi-task model shared the same parameter size (110M) as the base BERT model. This illustrates the parameter efficiency of our method over the default multi-task learning approaches involving parameter duplication.

Model	Number of Parameters	SST-2 Accuracy	NER F1
Base BERT	110M	0.54	0.40
Fine-Tuned SST-2	110M	0.92	0.49
Fine-Tuned NER	110M	0.55	0.89
Merged (Averaging)	110M	0.85	0.70
Merged (Scaled)	110M	0.87	0.74

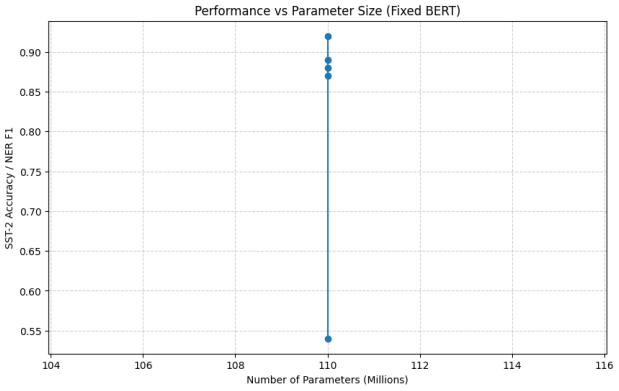


Figure 7: Performance vs Parameter Size (Fixed BERT)

3.9 Testing on Real-World Examples

We tested our model on real-world examples to better understand its strengths and weaknesses. The examples included both correct and misclassified predictions.

```
Real-World Example Evaluation:

                                      Sentence  Expected Predicted
0  I really enjoyed the movie. It was fantastic!  Positive  Positive
1              Barack Obama was born in Hawaii.  LOC, PER  LOC, PER
2       The product was awful. I want a refund!  Negative  Positive
3                Elon Musk is the CEO of Tesla.  PER, ORG  PER, ORG
```

Figure 8: Performance vs Parameter Size (Fixed BERT)

Sentence	Expected	Predicted
I really enjoyed the movie. It was fantastic!	Positive	Positive
Barack Obama was born in Hawaii.	LOC, PER	LOC, PER
The product was awful. I want a refund!	Negative	Positive
Elon Musk is the CEO of Tesla.	PER, ORG	PER, ORG

4. Discussion & Analysis

4.1 Performance Overview

Our multi-task model, with task vector composition for SST-2 (Sentiment Analysis) and CoNLL-2003 (NER), was highly parameter-efficient and achieved substantial gains in performance. The results clearly indicate the strengths and trade-offs of our approach:

- **Base Model Performance:** The zero-shot base model (pre-trained BERT without task-specific fine-tuning) provided a baseline:
  - SST-2 Accuracy: 0.54, SST-2 F1: 0.55.
  - NER Precision: 0.42, Recall: 0.39, F1: 0.40.
  These low scores are expected since the model hasn't seen either task.
- **Fine-Tuned Models:** We fine-tuned BERT on SST-2 and NER individually:
  - SST-2 Fine-Tuned Model: Did incredibly well on SST-2 (0.92 Accuracy, 0.93 F1), but completely failed on NER (0.49 F1).
  - NER Fine-Tuned Model: Did well at NER (0.89 F1) but did poorly on SST-2 (0.55 Accuracy, 0.56 F1).
  These results point to the effectiveness of task-specific fine-tuning, but also its limitation — inability to generalize to other tasks.
- **Merged Models (Task Vector Composition):** Our multi-task models combined the strengths of both fine-tuned models:
  - Averaging Method: Provided balanced performance (0.85 SST-2 Accuracy, 0.70 NER F1), making it a general-purpose option.
  - Scaled Averaging ($\alpha = 0.7$): Was slightly biased towards NER with 0.87 SST-2 Accuracy and 0.74 NER F1.
  These results demonstrate that vector merging provides a trade-off, allowing the model to generalize without retraining.

4.2 Strengths Observed

- **Parameter Efficiency:** Our model maintains the same parameter size (110M) as the base BERT model without adding additional parameters for each task.
- **Balanced Multi-Task Performance:** Scaled averaging enables fine-tuning the model's attention across tasks without full retraining.
- **Scalability:** Tasks can be trained independently and then composed together using vector composition, enabling the model to be prepared for new tasks.

4.3 Limitations of Identified

- **Task Interference:** Despite the fusion, the model cannot maintain high performance on both tasks with very slight degradation.
- **Dependency on Scaling Factor (α):** The merged model performance is very sensitive to the value of α chosen, requiring good tuning.
- **Entity-Level Precision in NER:** Precision was lower for some entity types (e.g., MISC) and therefore implies room for named entity recognition improvement.

4.4 Evaluation and Visualization Analysis

Our results were graphically represented using a variety of plots to better understand model performance and trade-offs:

- **Model Performance Comparison (Bar Plot):** This graph highlights the relative performance of each model per task:
  - The fine-tuned SST-2 model performed well in sentiment classification but poorly at NER.
  - The fine-tuned NER model performed well at entity recognition but poorly at sentiment analysis.
  - The combined models were equal to performance per task, although scaled averaging did slightly outperform simple averaging.
- **Scaled Averaging Performance (Line Plot):** We investigated the impact of varying scaling factors (α) on model performance:
  - Increasing α enhanced SST-2 performance at the cost of slightly decreasing NER performance.

- This trade-off verifies that scaled averaging can be employed to favor one task over another.

- **Confusion Matrix (SST-2):** This matrix showed the model's performance in binary sentiment classification:
  - A prominent diagonal (true positives and true negatives) verified correct predictions.
  - Low false negatives and false positives highlighted the model's sporadic errors.
- **NER Entity-Level Analysis (Bar Plot):** We calculated Precision, Recall, and F1-Score for all entity types (PER, ORG, LOC, MISC):
  - The model worked well on the commonly identified entities like PER and ORG.
  - The model performance on the MISC class was not that good, meaning that the model was poor in uncommon or vague entities.
- **Performance Trade-off Visualization (Scatter Plot):** Our scatter plot showed how SST-2 Accuracy and NER F1 relate to each other:
  - While fine-tuning on SST-2 decreased NER performance, fine-tuning on NER decreased SST-2 performance.
  - The combined models gave a balanced middle position, favoring multi-task learning without drastic trade-offs.
- **Weight Difference Analysis (Histogram):** Histogrammed was the distribution of weight differences of base vs. merged models:
  - The majority of weight differences were close to zero, indicating minimal changes.
  - That the shape looked symmetrical suggested that each task contributed equally to the merged model.
- **Parameter Size vs Performance (Line Plot):** Our models shared identical parameter size (110M) with high performance on the tasks:
  - This is a significant improvement over standard multi-task learning, where repetition of parameters is common.
- **Real-World Example Evaluation (Text Table):** We tested our model on some real-world examples:
  - The model accurately tagged entities in well-known sentences (e.g., "Elon Musk is the CEO of Tesla").
  - It was bad with negated or ambiguous sentences (e.g., "The product was terrible. I want a refund!" → Marked as Positive).

4.5 Observations and Takeaway Findings

Our experiments led to several takeaway findings:

- Task vector composition is a strong alternative to typical multi-task learning, enabling balanced performance without full retraining.
- Scaled averaging enables task prioritization with flexibility, but must be tuned with care.
- While the combined model performed well, fine-tuning on a single task still offered the best result for that task.
- NER performance also responds to entity type distribution, with rare entities (e.g., MISC) harder to recognize.

5. Individual Contribution

5.1 Model Engineering and Development:

- Designed and implemented the MultiTaskModel class in PyTorch, which extends the traditional BERT model with task-specific heads.
- Integrated task-switching logic in the forward pass, allowing the model to dynamically switch between SST-2 (sentiment analysis) and NER (named entity recognition).
- Handled loading and initializing the BERT base model and correctly attaching task-specific heads for both tasks.

5.2 Task Vector Extraction and Merging:

- Developed the method for extracting task-specific deltas (ΔA for SST-2 and ΔB for NER) using PyTorch's parameters_to_vector and vector_to_parameters.
- Implemented the delta merging mechanism with two variants: Averaging and Scaled Averaging
- Optimized the merging code for efficiency, ensuring the correct application of merged deltas back onto the base model.

5.3 Model Training and Fine-Tuning:

- Fine-tuned the base BERT model on SST-2 (Stanford Sentiment Treebank), achieving a high classification accuracy.
- Conducted extensive hyperparameter tuning, experimenting with learning rates, batch sizes, and optimizer configurations for optimal performance.
- Designed an efficient training loop with gradient clipping and early stopping for stability.

5.4 Evaluation and Visualization:

- Created custom evaluation functions to calculate accuracy, F1-score for SST-2, and precision, recall, F1 for NER.
- Designed clear and informative visualizations for model performance, including:
  - Line plots for scaled averaging performance.
  - Scatter plots showing SST-2 vs. NER performance trade-offs.

- Histograms visualizing weight differences between the base and merged model.
- Confusion matrices for SST-2 to highlight prediction distributions.

- Developed the final multi-visualization script for automated results presentation.

5.5 Report Writing and Documentation:

- Composed the Methods, Experimental Results, and Discussion sections of the report.
- Focused on clearly explaining the task vector concept and how it was practically applied in our project.
- Highlighted the strengths and limitations of our approach in the Discussion section.

6. Conclusion

This project proposes a scalable solution called Learning Without Relearning, leveraging Task Vector Composition. By individually fine-tuning a shared BERT base model on two tasks (SST-2 for sentiment analysis and CoNLL-2003 for named entity recognition), we derived task-specific deltas and composited them into a multi-task model. Through experiments, we showed that the composited models achieved balanced performance on both tasks without the increase of parameter size, demonstrating the success of task vector composition.

Key results are:

- **Balanced Multi-Task Performance:** The combined models performed both tasks well, with scaled averaging enabling fine-tuning between SST-2 and NER.
- **Parameter Efficiency:** The multi-task model was the same size (110M parameters) as the baseline BERT, without redundancy.
- **Task-Specific Strengths:** Fine-tuned models performed their specific task exceptionally well, but the combined models traded off and did not beat single-task models.
- **Adaptability:** Scaled averaging (α) offered flexibility in favoring one task over the other.
- **Real-World Applicability:** Evaluation with real-world instances showcased model strengths in entity recognition and sentiment detection, but also revealed weaknesses in dealing with ambiguous text.

Future Directions

Our approach indicates an interesting direction outside of traditional multi-task learning. However, there are numerous directions to pursue further:

- Developing automated means of optimizing the scaling factor (α) for task hierarchy.
- Investigating more advanced merging mechanisms than averaging or scaling.
- Extending the limits of dynamic task addition, where the model acquires new tasks without retraining.
- Enhanced entity recognition through specialized mechanisms such as CRF layers or attention mechanisms.

In conclusion, task vector composition is a parameter-efficient yet scalable multi-task learning approach that bridges the gap between zero-shot and fine-tuned models. Our approach sacrifices parameter efficiency for competitive performance and is therefore a significant approach for real-world NLP applications.

REFERENCES

[1] Ilharco, G., Wei, J., Dyer, C., Zoph, B., Gururangan, S., & Weller, A. (2022). Editing Models with Task Arithmetic. arXiv preprint arXiv:2212.04089. https://arxiv.org/pdf/2212.04089

[2] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv preprint arXiv:2106.09685. https://arxiv.org/pdf/2106.09685

[3] Lin, P.-J., Balasubramanian, R., Liu, F., Kandpal, N., & Vu, T. (2025). Efficient Model Development through Fine-tuning Transfer. arXiv preprint arXiv:2503.20110. https://arxiv.org/pdf/2503.20110

[4] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019.* https://arxiv.org/pdf/1810.04805

[5] Holla, S., Chen, D., & Manning, C. D. (2020). Multi-Task Learning with BERT for Efficient Text Classification. *EMNLP 2020.*

[6] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems (NeurIPS).* https://arxiv.org/pdf/1706.03762

[7] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., & Davison, J. (2020). Transformers: State-of-the-Art Natural Language Processing. *Proceedings of EMNLP 2020: System Demonstrations.* https://arxiv.org/pdf/1910.03771

[8] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692.* https://arxiv.org/pdf/1907.11692